

PROJETO Nº 2 · MATRIZ DE PROJETOS EDGE AI / VISÃO COMPUTACIONAL

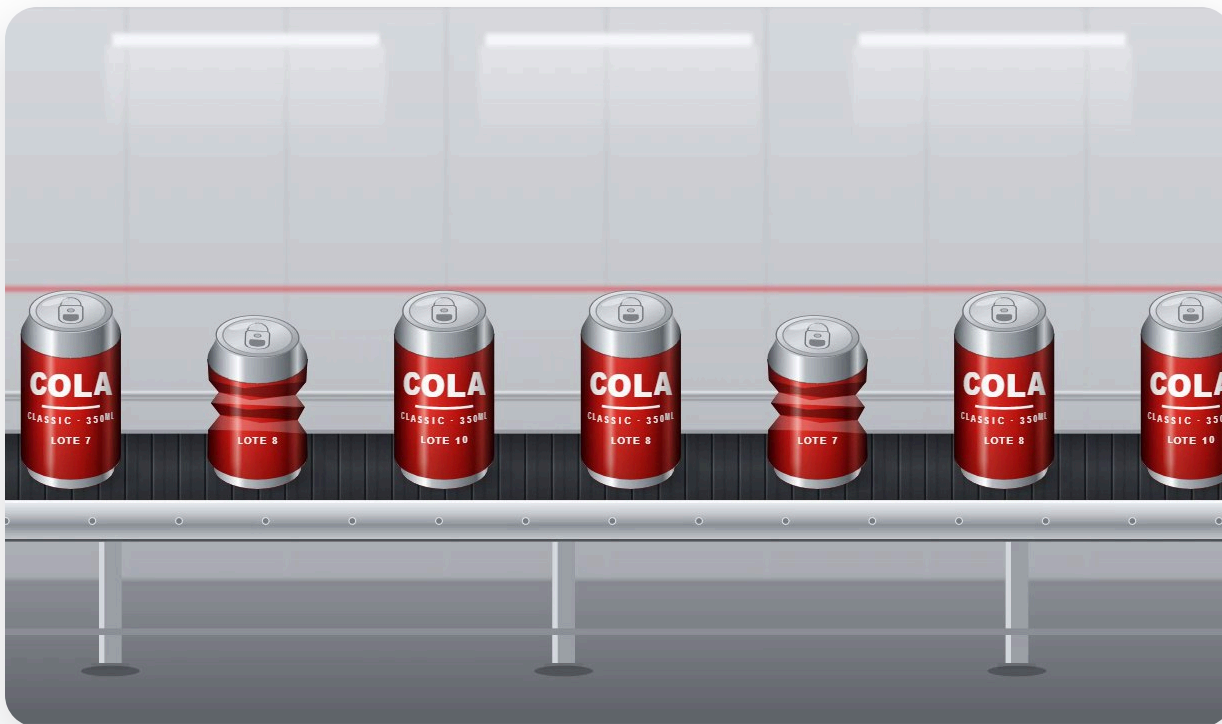
Inspeção Visual Automática de Qualidade em Linha de Produção

Latas conformes e não conformes, em tempo real, na Raspberry Pi 5

Francisco Lima

Banca técnica e equipe pedagógica

A inspeção manual é lenta, cara e sujeita à fadiga



1,9

latas por segundo

na esteira do vídeo de teste

40%

não conformes

64 das 160 latas do vídeo de teste

100%

das latas inspecionadas

sem interrupção do fluxo produtivo

Inspeção manual é lenta, cara e cansa o inspetor. No vídeo: 1,9 latas/s e 40% não conformes. Meta da Matriz: inspecionar 100% sem parar o fluxo.

Motivação da Matriz de Projetos, projeto nº 2 · quadro real do vídeo da esteira

O que o projeto pede

Um sistema embarcado de inspeção visual na **Raspberry Pi 5** que classifique as peças como **conformes ou não conformes em tempo real**, sem interrupção do fluxo produtivo.



Capturar

Imagens das peças em movimento



Inferência

Modelo de detecção/classificação de defeitos



Classificar

Conformes ou não conformes, em tempo real



Sinalizar

Peças reprovadas via saída digital (GPIO ou relay)



Registrar

Imagens anotadas de não conformidades, para rastreabilidade



Embarcado

Na Raspberry Pi 5, antes do empacotamento

Pedidos da Matriz: capturar em movimento, inferir com modelo de defeitos, classificar conforme/não conforme em tempo real, sinalizar por GPIO/relay e registrar imagens anotadas.

Neste projeto as peças são latas: lata_ok (conforme) · amassada, sem_rotulo, sem_cor (não conformes)

Como cada item pedido foi atendido

PEDIDO NA MATRIZ	NO PROJETO
Capturar imagens das peças em movimento	OpenCV lê a câmera USB ou o vídeo da esteira, quadro a quadro
Modelo de detecção/classificação de defeitos	YOLO26n pré-treinado (COCO) com ajuste fino em 4 classes · mAP50 0,93
Conformes ou não conformes	lata_ok = conforme · amassada, sem_rotulo, sem_cor = não conforme
Em tempo real, sem interrupção do fluxo	~67 ms por imagem na Pi 5 (referência Ultralytics, NCNN) · uma lata a cada ~525 ms
Sinalizar via saída digital (GPIO ou relay)	Pulso de 0,2 s no GPIO 17 para o relé do soprador (simulado fora da Raspberry)
Imagens anotadas de não conformidades	saida/defeitos/ com a lata marcada · inspecao.csv com uma linha por lata

Cada pedido tem resposta no projeto. Os 67 ms são referência da Ultralytics: os testes foram no PC e em emulação ARM64; o GPIO é simulado fora da Pi.

Da imagem da lata à sinalização da reprovada



mAP50 0,93

validação, 4 classes

~67 ms

por imagem na Pi 5 (ref. Ultralytics)

160 latas

96 conformes · 64 não conformes

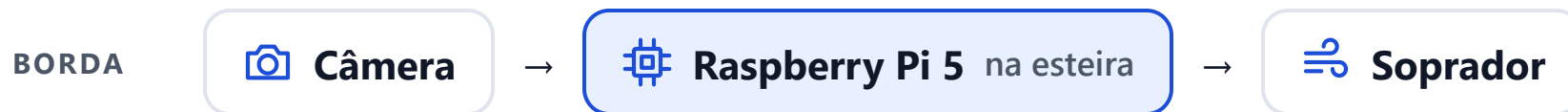
API + Docker

linux/amd64 e linux/arm64

Captura → YOLO26n detecta → rastreio numera → na linha virtual a lata é classificada uma vez → não conforme aciona o GPIO 17 → imagem anotada vai para o registro.

Código, README e benchmark: github.com/geanusksama/inspecao-latas

O modelo vai até a máquina, não a imagem até a nuvem



Edge AI: o modelo roda no próprio equipamento, ao lado da esteira. A imagem é processada onde foi capturada: **~67 ms por imagem** na Pi 5 (ref. Ultralytics).



Nuvem: a imagem viaja pela internet até um servidor remoto, que processa e devolve a resposta.



Latência

Tempo entre capturar a imagem e sair a decisão. Na borda não há ida e volta pela internet.



Sem depender da rede

O sistema funciona sem internet: se a rede cair, a linha continua inspecionando.



Privacidade e custo

As imagens ficam na fábrica e não há cobrança por uso de servidor na nuvem.

Edge AI = modelo rodando no próprio equipamento. Ganhos: latência baixa, funciona sem internet, imagens ficam na fábrica. Limite: sem GPU, por isso modelo nano + NCNN.

Limite da borda: CPU ARM sem GPU · por isso modelo nano + formato NCNN

Para construir o modelo (no PC com GPU)



Python + venv

Linguagem do projeto; o venv isola as versões das bibliotecas

No projeto: scripts numerados em scripts/



PyTorch

Biblioteca de redes neurais: faz os cálculos do modelo e do treino

No projeto: 2.14.1, com CUDA no PC e só CPU no container



GPU NVIDIA + CUDA

A placa de vídeo calcula em paralelo; o CUDA liga o PyTorch a ela

No projeto: treino em ~94 s na GTX 1650



Ultralytics YOLO26

Detector de objetos em uma passada; a biblioteca treina, detecta, rastreia e exporta

No projeto: YOLO26n (nano), pré-treinado no COCO



OpenCV

Biblioteca de visão computacional: vídeo, câmera, desenho e imagens

No projeto: captura, anotação e tela da inspeção



Dataset próprio

Imagens com rótulos (caixa + classe) que ensinam o modelo

No projeto: 60 imagens, 4 classes, 48 treino / 12 validação

Para levar o modelo até a linha



NCNN

Motor de inferência leve, feito para processadores ARM

No projeto: 67 ms na Pi 5, contra 299 ms do PyTorch (ref. Ultralytics)



ByteTrack

Rastreador: liga as detecções de um quadro ao seguinte e mantém o número de cada objeto

No projeto: cada lata é contada uma vez



Raspberry Pi 5

Computador de placa única, CPU ARM64 (Cortex-A76), sem GPU dedicada

No projeto: roda a inspeção ao lado da esteira



GPIO + relé

GPIO: pino digital da placa.
Relé: chave elétrica acionada por esse pino

No projeto: GPIO 17, pulso de 0,2 s para o soprador



FastAPI

Biblioteca Python para criar APIs HTTP, com página de teste (Swagger)

No projeto: /detectar (JSON) e /detectar/imagem (PNG)



Docker

Empacota código, bibliotecas e modelo numa imagem que roda igual em qualquer máquina

No projeto: linux/amd64 (PC) e linux/arm64 (Pi)

Na linha: NCNN (formato para ARM), ByteTrack (segue as latas), Raspberry Pi 5, GPIO + relé (soprador), FastAPI (API) e Docker (mesma imagem no PC e na Pi).

Dockerfile e docker-compose na raiz do repositório

Do zero ao sistema na linha, em 17 passos

Ambiente

- 1 Instalar
- 2 Testar a GPU

Usar o YOLO

- 3 Detectar numa imagem
- 4 Ler um vídeo
- 5 Ler a câmara
- 6 Onde ficam os resultados

Modelo próprio

- 7 Por que um dataset
- 8 Capturar imagens
- 9 Anotar
- 10 Separar
- 11 Treinar

Embarcar

- 12 Exportar para NCNN
- 13 Medir no hardware

Na linha

- 14 Rastrear e classificar
- 15 Sinalizar e registrar
- 16 API HTTP
- 17 Docker

Cada passo: **um bloco de código curto**, para que serve e por que importa. No fim, **o passo a passo para executar e testar**.

Ambiente isolado, PyTorch com CUDA, depois o YOLO

terminal (Windows)

Linux/Pi: source venv/bin/activate

```
python -m venv venv          # cria o ambiente isolado
venv\Scripts\activate        # entra nele
pip install torch==2.14.1 torchvision==0.29.1 --index-url https://download.pytorch.org/whl/cu130
pip install -r requirements.txt # ultralytics, opencv-python...
```

venv

As versões do projeto não brigam com outros programas do computador

PyTorch primeiro

No Windows, se o ultralytics vier antes, ele instala um PyTorch só de CPU

ultralytics

Traz o YOLO26 e o comando `yolo`; opencv abre vídeo e janelas

docs.ultralytics.com/quickstart pytorch.org/get-started/locally

Ambiente virtual, depois PyTorch com CUDA, depois requirements.txt. No Windows, se o ultralytics vier antes, ele instala PyTorch só de CPU.

Versões do projeto: requirements.txt (Python 3.14, CUDA 13.0)

O PyTorch está enxergando a placa de vídeo?

python scripts/0_verificar_gpu.py

```
import torch

print(torch.__version__)          # +cu130?
print(torch.cuda.is_available())  # True = GPU ok
print(torch.cuda.get_device_name())
```

saída esperada

```
2.14.1+cu130
True
NVIDIA GeForce GTX 1650
```

True: o treino vai usar a GPU (`device=0`)

False: foi instalado o PyTorch de CPU. Reinstale com o índice `cu130`

`nvidia-smi` no terminal mostra o driver e a memória da placa

A GPU só é usada para **treinar**. Na Raspberry, tudo roda na CPU

Tem que aparecer +cu130, True e o nome da placa. False = PyTorch de CPU. A GPU só é usada no treino (~94 s na GTX 1650).

Treino do projeto: 60 imagens, 30 épocas, ~94 s na GTX 1650

Três linhas e o YOLO já enxerga objetos

python

exemplo mínimo

```
from ultralytics import YOLO

modelo = YOLO("yolo26n.pt")    # baixa na 1ª vez
resultados = modelo.predict("esteira.jpg", save=True)

for caixa in resultados[0].boxes:
    nome = modelo.names[int(caixa.cls)]
    print(nome, float(caixa.conf), caixa.xyxy.tolist())
```

yolo26n.pt: versão *nano*, pré-treinada com 80 classes do COCO (pessoa, garrafa, carro...)

save=True: grava a imagem com as caixas desenhadas

boxes: cada objeto achado, com classe, confiança e caixa

Sem Python: `yolo predict model=yolo26n.pt source=esteira.jpg`

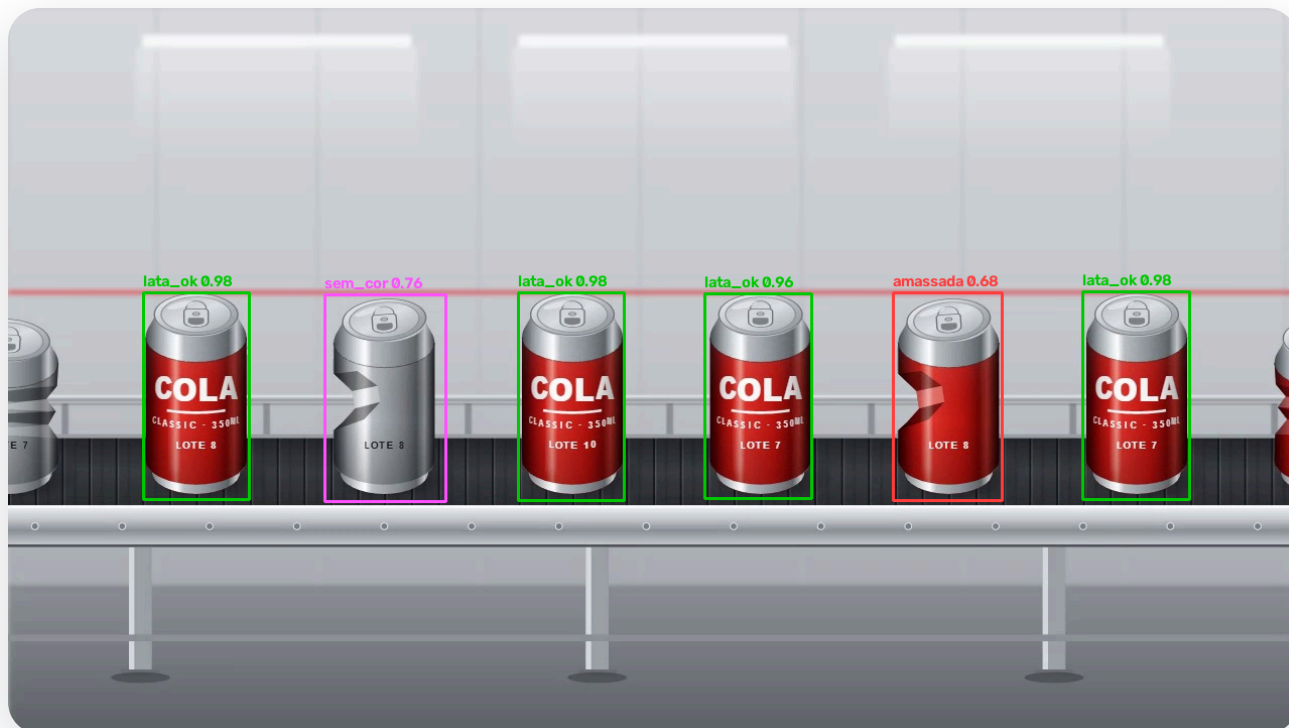
POR QUE IMPORTA

É o mesmo `predict` que a API usa. Mas o modelo pronto não sabe o que é "lata amassada": isso vem no passo 7.

yolo26n.pt já conhece 80 classes do COCO. predict() devolve as caixas. É o mesmo predict da API, mas o modelo pronto não conhece lata amassada.

docs.ultralytics.com/modes/predict · docs.ultralytics.com/models/yolo26

Cada detecção tem caixa, classe e confiança



Caixa: onde a lata está (x1, y1, x2, y2 em pixels)

Classe: lata_ok · amassada · sem_rotulo ·
sem_cor

Confiança: de 0 a 1, quanta certeza o modelo tem

Limiar 0,5: abaixo disso a detecção é descartada (conf=0.5)

Caixa = onde está a lata. Classe = qual o defeito. Confiança = certeza de 0 a 1. Abaixo de 0,5 a detecção é descartada.

Resposta real do modelo treinado (endpoint /detectar/imagem)

Um vídeo é uma sequência de imagens

python

exemplo

```
import cv2
from ultralytics import YOLO

modelo = YOLO("yolo26n.pt")
captura = cv2.VideoCapture("esteira.mp4")
while True:
    ok, quadro = captura.read()    # próximo quadro
    if not ok:                    # acabou o vídeo
        break

    resultado = modelo.predict(quadro)[0]
    cv2.imshow("YOLO", resultado.plot())
    if cv2.waitKey(1) == 27:      # ESC sai
        break
```

Em amarelo: o que mudou em relação à imagem

VideoCapture abre o vídeo; **read()** entrega um quadro por vez

plot() devolve o quadro já com as caixas desenhadas

Atalho do Ultralytics: `predict("esteira.mp4", show=True)`

Vídeo = sequência de imagens. VideoCapture abre, read() entrega um quadro por vez. Esse laço é a base da inspeção.

O laço com OpenCV dá controle de cada quadro: é a base da inspeção (6_inspecionar.py)

Do vídeo para a câmera: muda uma linha

python

exemplo

```
captura = cv2.VideoCapture(0)  # 0 = 1ª câmera USB
while True:
    ok, quadro = captura.read()
    ...                        # o resto é igual
```

terminal

no projeto

```
python scripts/6_inspecionar_basico.py esteira.mp4
python scripts/6_inspecionar_basico.py 0
```

Imagem, vídeo ou câmera: para o modelo, tudo é um quadro

0, 1, 2... são as câmeras ligadas ao computador

No menu do projeto: **Enter** = vídeo de demonstração, **0** = câmera USB

POR QUE IMPORTA

Desenvolvemos com o vídeo gravado (repetível) e, na linha, trocamos só a fonte.

Toda execução cria uma pasta nova

`predict(save=True)`

`runs/detect/`

- └ `predict/` *1ª execução*
 - | └ `esteira.jpg` *com as caixas*
 - | └ `labels/` *save_txt=True*
 - | └ `crops/` *save_crop=True*
 - | └ `predict2/` *a seguinte...*

`project=` e `name=` escolhem a pasta; `exist_ok=True` reaproveita.

`train(project="treinos", name="latas")`

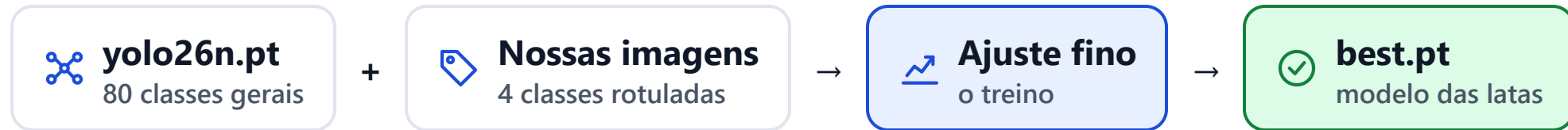
`treinos/latas/`

- └ `weights/`
 - | └ `best.pt` *o melhor: é o usado*
 - | └ `last.pt` *última época*
- └ `results.png` *curvas do treino*
- └ `results.csv` *métricas por época*
- └ `confusion_matrix.png`
- └ `val_batch0_pred.jpg`
- └ `args.yaml` *parâmetros usados*

`predict(save=True)` grava em `runs/detect/predict`. O treino do projeto grava em `treinos/latas`: `best.pt`, curvas e matriz de confusão.

Pastas reais do projeto: `treinos/latas/`

O modelo pronto não conhece lata amassada



Baixar um pronto (para estudar)

COCO · Open Images · ImageNet
Roboflow Universe · universe.roboflow.com
Kaggle · kaggle.com/datasets

Criar o próprio (o que fizemos)

Vídeo de treino da **própria esteira** (vtrieino/curto.mp4):
mesma câmera, luz e fundo do vídeo de teste.

POR QUE IMPORTA

Dataset = imagens + rótulos. A qualidade do modelo depende mais dos dados do que do código.

Um vídeo curto vira dezenas de imagens

```
python scripts/1_capturar.py (resumido)

import cv2

captura = cv2.VideoCapture("vtrieino/curto.mp4")
n = 0
while True:
    ok, quadro = captura.read()
    if not ok:
        break
    if n % 10 == 0:                # 1 a cada 10 quadros
        nome = f"dataset/imagens/curto_{n:06d}.jpg"
        cv2.imwrite(nome, quadro)
    n += 1
```

30 fps ÷ 10 = 3 imagens por segundo de vídeo

Quadros vizinhos são quase iguais: pular evita imagens repetidas

Varie **posição, luz e tipo de defeito**

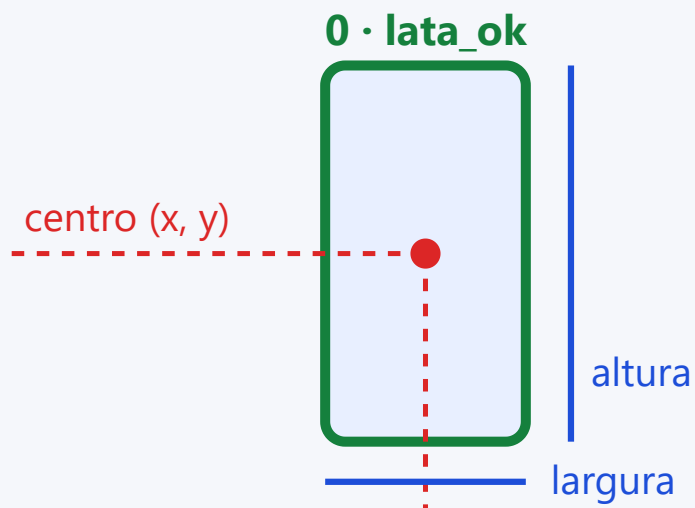
No projeto: **60 imagens** com as 4 classes

1 quadro a cada 10 = 3 imagens por segundo, sem repetir quase a mesma imagem. No projeto: 60 imagens.

Próximo passo: marcar as latas em cada imagem

Uma caixa e uma classe para cada lata

imagem (largura e altura = 1)



Uma linha por lata no arquivo `curto_000000.txt`:

0	0.287	0.531	0.088	0.275
classe	x centro	y centro	largura	altura

Valores de 0 a 1: valem para qualquer resolução

2_annotar.py: arrasta o mouse, teclas 1 a 4 escolhem a classe

Alternativas: Label Studio, CVAT, Roboflow

ATALHO

Treinou com poucas? O `5_prerotular.py` marca o resto sozinho e você só corrige.

Guardar imagens que o modelo nunca viu

scripts/3_separar.py cria

```
dataset/yolo/  
├─ images/  
│   ├── train/    48 imagens  
│   └── val/      12 imagens  
├─ labels/  
│   ├── train/    48 .txt  
│   └── val/      12 .txt  
└─ data.yaml
```

yaml

data.yaml

```
path: d:/YOLOENTREGA/dataset/yolo  
train: images/train  
val: images/val  
names:  
  0: lata_ok  
  1: amassada  
  2: sem_rotulo  
  3: sem_cor
```

POR QUE IMPORTA

Medir com as mesmas imagens do treino é como dar a prova com o gabarito: 20% ficam só para a validação.

Ajuste fino: ensinar só as 4 classes

```
python scripts/4_treinar.py (resumido)

from ultralytics import YOLO

modelo = YOLO("yolo26n.pt") # parte do que já sabe
modelo.train(
    data="dataset/yolo/data.yaml",
    epochs=30,           # passadas por todas as imagens
    imgsz=640,           # tamanho da imagem no treino
    batch=8,             # imagens por passo
    device=0,            # 0 = GPU · "cpu" = sem GPU
    project="treinos", name="latas",
)
```

0,93**mAP50**

nota geral da detecção

0,91**Recall**

das latas, quantas achou

0,83**Precisão**

do que achou, quanto acertou

5 MB**best.pt**

o modelo treinado

O mesmo modelo, no formato da Raspberry

python scripts/8_exportar_ncnn.py (resumido)

```
from ultralytics import YOLO

YOLO("treinos/latas/weights/best.pt").export(
    format="ncnn", imgsz=640)

# depois, usa igual ao .pt:
modelo = YOLO("treinos/latas/weights/best_ncnn_model")
```

pasta gerada

```
best_ncnn_model/
├─ model.ncnn.param  estrutura da rede
├─ model.ncnn.bin    pesos aprendidos
└─ metadata.yaml     classes e tamanho
```

NCNN: motor de inferência feito para processadores ARM, como o da Pi

Recomendado pelo guia Raspberry Pi da Ultralytics

Outros formatos: ONNX, OpenVINO, TFLite

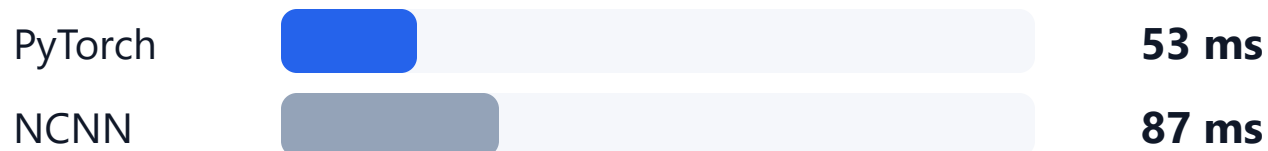
INT8 (próximo passo): menor e mais rápido, com pequena perda de precisão

Uma linha exporta para NCNN: estrutura, pesos e classes. Usa igual ao .pt. NCNN é feito para processadores ARM.

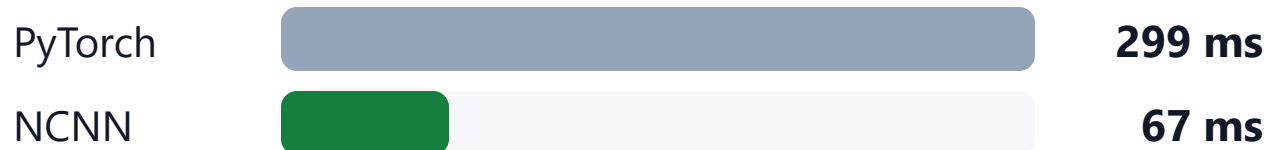
docs.ultralytics.com/integrations/ncnn · docs.ultralytics.com/modes/export

O mesmo modelo, velocidades diferentes

Notebook (Intel i5, CPU) · medido



Raspberry Pi 5 (ARM) · referência oficial



python scripts/9_benchmark.py (ideia)

```
inicio = time.perf_counter()
modelo.predict(quadro)
ms = (time.perf_counter() - inicio) * 1000
```

Meça onde vai rodar

No notebook, o NCNN perde. Na Raspberry, ganha 4,5×. O número do PC não vale para a placa.

Próxima lata a cada **525 ms**

Decisão em **67 ms**

Folga de **8×**

No notebook o NCNN perdeu (87×53 ms). Na Pi 5 ganha (67×299 ms, ref. Ultralytics). Medir no hardware alvo. Folga de 8× por lata.

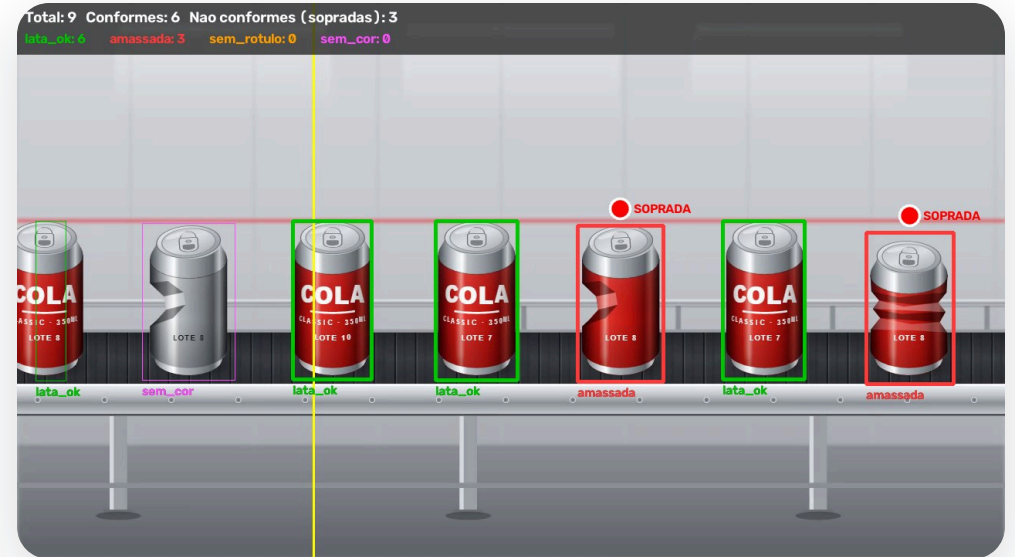
Notebook: mediana de 100 quadros (docs/benchmark.md) · Pi 5: docs.ultralytics.com/guides/raspberry-pi

Cada lata é classificada uma vez, ao cruzar a linha

python scripts/inspecao.py (trechos)

```
# detectar(): detecta e segue as latas
caixas = modelo.track(quadro, persist=True,
                      conf=CONFIANCA, agnostic_nms=True,
                      tracker="bytetrack.yaml")[0].boxes

# cruzou(): a lata passou pela linha?
def cruzou(x_antes, x_agora, linha):
    return (x_antes < linha <= x_agora
            or x_agora <= linha < x_antes)
```



track(): a lata mantém o mesmo número entre quadros

Na linha: recebe a classe mais vista nos quadros e é contada uma vez

track() mantém o número da lata. Ao cruzar a linha, ela recebe a classe mais vista e é contada uma vez. agnostic_nms: 160 latas, não 164.

agnostic_nms: uma caixa por lata (sem ele, 164 latas em vez de 160) · docs.ultralytics.com/modes/track

A lata reprovada é sinalizada e registrada

```
python scripts/saida_gpio.py (resumido)

from gpiozero import OutputDevice # só na Pi

soprador = OutputDevice(17)      # GPIO 17 (BCM)

def soprar():
    # pulso de 0,2 s sem travar a inspeção
    soprador.blink(on_time=0.2, off_time=0,
                   n=1, background=True)
```

saida/

```
saida/
├ inspecao.csv      1 linha por lata
├ resumo.txt        totais por classe
├ defeitos/         quadro marcado
│   └ 00002_amassada.jpg
└ recortes/         só a lata
    ├── amassada/
    ├── sem_rotulo/
    └ sem_cor/
```

POR QUE IMPORTA

GPIO 17 → relé → soprador. No PC, sem GPIO, a saída é simulada e o mesmo código roda. O CSV e as imagens anotadas dão a rastreabilidade de cada não conformidade.

Qualquer sistema manda a foto e recebe a resposta

python

scripts/api.py (resumido)

```
from fastapi import FastAPI, File, UploadFile

app = FastAPI(title="Inspeção de Latas")

@app.get("/saude")
def saude():
    return {"status": "ok"}

@app.post("/detectar")
def detectar(arquivo: UploadFile = File(...)):
    imagem = ler_imagem(arquivo)    # bytes → imagem
    return para_json(inferir(imagem), imagem)
```

GET /saude: o serviço está no ar?

POST /detectar: JSON com classe, confiança, caixa e resumo

POST /detectar/imagem: PNG com as caixas

Erros: 400 (não é imagem) · 413 (maior que 10 MB)

testar

```
# navegador: localhost:8000/docs
curl -F "arquivo=@lata.jpg" \
    localhost:8000/detectar
```

/saude, /detectar (JSON) e /detectar/imagem (PNG). Erros 400 e 413. Demo: localhost:8000/docs → Try it out → Execute.

fastapi.tiangolo.com · o modelo é carregado uma vez, quando a API sobe

Uma receita, a mesma imagem no PC e na Pi

Dockerfile (resumido)

```
FROM python:3.12-slim-bookworm
RUN pip install torch torchvision \
    --index-url https://download.pytorch.org/whl/cpu
COPY requirements-docker.txt .
RUN pip install -r requirements-docker.txt
COPY scripts/ scripts/
COPY treinos/latas/weights/best_ncnn_model/ ...
EXPOSE 8000
CMD ["uvicorn", "api:app", ...] # sobe a API
```

terminal

```
docker compose up -d --build
docker buildx build \
    --platform linux/amd64,linux/arm64 \
    -t inspecao-latas .
```

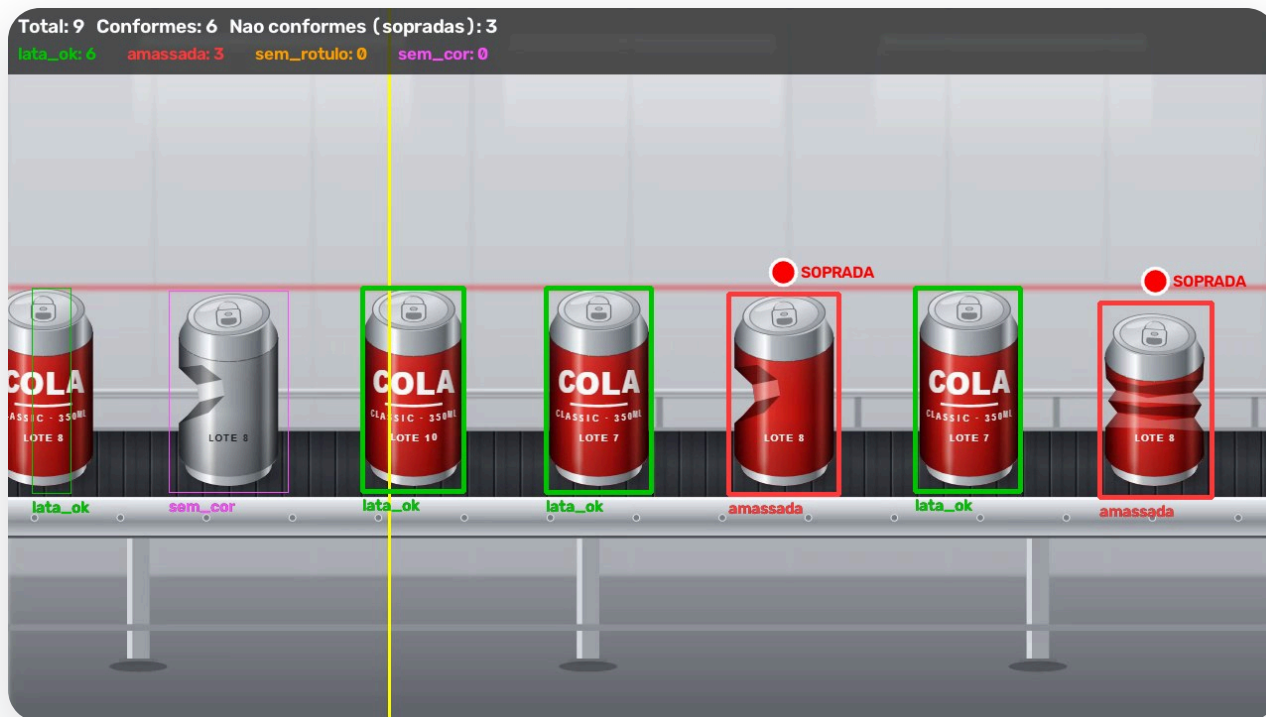
Imagem = a receita pronta · **container** = ela rodando

compose: porta 8000, `restart: always`, pasta `saida/` fora do container

Na Pi: `privileged` para acessar o GPIO

RESULTADO

O sistema completo no vídeo de teste



160

latas inspecionadas

igual à contagem de referência

96 · 64

conformes · não conformes

amassada 32 · sem_cor 24 · sem_rotulo 8

8×

de folga estimada na Pi 5

67 ms por imagem, uma lata a cada 525 ms

160 latas (igual à referência): 96 conformes e 64 não conformes. Folga estimada de 8× na Pi 5. Falta validar na placa física.

Tela real do 6_inspecionar.py · saida/resumo.txt · esteirafull.mp4

Passo a passo para qualquer pessoa rodar o projeto

No PC

- 1 Instalar o Docker
- 2 Clonar e subir o container
- 3 Testar a API
- 4 Inspecionar o vídeo

Docker

- 5 Comandos mínimos

Raspberry Pi 5

- 6 Instalar e subir na placa
- 7 Conferir na placa

Código

- 8 Rodar sem Docker
- 9 Subir para o Git
- 10 Problemas comuns

Guia completo no repositório: `COMO_TESTAR.md`

O que instalar antes (uma vez)



Docker Desktop

docs.docker.com/desktop · abra e espere **Engine running**



Git (opcional)

git-scm.com · ou baixe o ZIP pelo botão verde **Code** no GitHub



Computador

Windows, Linux ou Mac · 8 GB de RAM · 15 GB livres



Internet só na 1ª vez

A montagem baixa ~1 GB. Depois roda sem internet

terminal · conferir se está tudo instalado

```
docker --version           # Docker version 2x.x...
docker compose version     # Docker Compose version v2...
git --version              # git version 2.x (se for usar o Git)
```

Docker Desktop aberto (Engine running). Git é opcional. Conferir com `docker --version`.

Não precisa de Python nem de GPU para testar: tudo vem dentro do container

Três comandos e a API está no ar

PowerShell (Windows) · igual no Linux e Mac

```
# 1. clonar o repositório do GitHub
git clone https://github.com/geanusksama/inspecao-latas.git
# 2. entrar na pasta (a que tem o Dockerfile)
cd inspecao-latas
# 3. montar a imagem e subir o container
docker compose up -d --build
```

- 1ª vez: **5 a 20 min** (baixa e monta). Depois, segundos
- Terminou quando aparece **Started**
- Abra **<http://localhost:8000/docs>**
- Espere ~1 min se a página não abrir: o modelo está carregando

O QUE ACONTECEU

O Docker leu o **Dockerfile**, instalou Python, PyTorch (CPU), o modelo NCNN e subiu a API na porta 8000.

git clone → cd inspecao-latas → docker compose up -d --build → localhost:8000/docs.

Parar: docker compose down · subir de novo: docker compose up -d

Mandar uma foto e ler a resposta

- 1 Abra **localhost:8000/docs** (Swagger)
- 2 **POST /detectar** → Try it out
- 3 Escolha uma imagem de `dataset/imagens`
- 4 **Execute** → leia o **Response body**
- 5 Repita em **POST /detectar/imagem**: volta a foto marcada

ou pelo terminal

```
curl localhost:8000/saude  
curl -F "arquivo=@foto.jpg" localhost:8000/detectar
```

resposta real (curto_000090.jpg)

JSON

```
{  
  "deteccoes": [  
    { "classe": "lata_ok",  
      "confianca": 0.982,  
      "conforme": true,  
      "caixa": { "x1": 134, "y1": 282, ... } },  
    ...  
  ],  
  "resumo": { "total": 6,  
              "conformes": 4,  
              "nao_conformes": 2 },  
  "metadados": { "formato": "ncnn", ... }  
}
```

Swagger: POST /detectar → Try it out → imagem → Execute. Resposta real: 6 latas, 4 conformes, 2 não conformes.

Cores na imagem: verde = boa · vermelho = amassada · laranja = sem rótulo · rosa = sem cor

O teste completo: contar as 160 latas

terminal · abre o menu dentro do container

```
docker compose exec inspecao python scripts/menu.py
```

resultado esperado (opção 1)

```
Lata 1: lata_ok
Lata 2: amassada
Lata 2: SOPROU via GPIO/Rele
...
Total de latas: 160
Conformes: 96
Nao conformes: 64
```

- 1 1 + Enter + Enter: inspeciona o vídeo de demonstração (~6 min no PC)
- 2 2: relatório e lotes com defeito (**lote 7: 24 · lote 8: 40**)
- 3 0: sai do menu
- 4 Resultados na pasta `saida/`: `inspecao.csv` (abre no Excel), `defeitos/`, `recortes/`

Os comandos mínimos do Docker

COMANDO	PARA QUE SERVE
<code>docker compose up -d --build</code>	Monta a imagem (só o que mudou) e liga em segundo plano
<code>docker compose ps</code>	Mostra se o container está rodando e saudável
<code>docker compose logs -f</code>	Acompanha as mensagens da API (Ctrl+C sai)
<code>docker compose exec inspecao python scripts/menu.py</code>	Roda um comando dentro do container
<code>docker compose restart</code>	Reinicia sem montar de novo
<code>docker compose down</code>	Desliga e remove o container (os resultados em saida/ ficam)
<code>docker images</code>	Lista as imagens montadas no computador
<code>docker system prune</code>	Apaga o que não está em uso e libera disco

up sobe, ps mostra, logs acompanha, exec roda dentro, down desliga. Na Pi: com sudo, na pasta raspberry/.

Na Raspberry, os mesmos comandos com sudo, dentro da pasta raspberry/

Da placa nova ao sistema ligando sozinho

- 1 Grave o **Raspberry Pi OS 64-bit** com o **Pi Imager**
- 2 Nas configurações: usuário, senha, Wi-Fi e **SSH**
- 3 Entre pelo PC com `ssh`
- 4 Baixe o projeto e rode o `iniciar.sh`
- 5 Relé ou LED: **GPIO 17** (pino 11) e **GND** (pino 6)

no PC, depois na Raspberry

```
ssh pi@inspecao.local          # ou ssh pi@IP-da-placa

sudo apt update && sudo apt install -y git
git clone https://github.com/geanusksama/inspecao-latas.git
cd inspecao-latas
sh raspberry/iniciar.sh        # instala Docker e liga

# menu de inspeção (opções 1 e 2, como no PC)
cd ~/inspecao-latas/raspberry
sudo docker compose exec inspecao python scripts/menu.py
```

O QUE O INICIAR.SH FAZ

Instala o Docker (1ª vez), monta a imagem arm64, liga com `restart: always` e mostra `http://IP:8000/docs`.

Pi OS 64 bits + SSH → git clone → sh raspberry/iniciar.sh (instala o Docker e sobe com restart: always).

raspberry/LEIAME.md · resultados para o PC: scp -r pi@inspecao.local:~/inspecao-latas/saida .

Lista de aceite na Raspberry Pi 5

- ☐ `http://IP:8000/saude` responde "**formato**": "**ncnn**"
- ☐ Menu, opção 1: **160 latas · 96 conformes · 64 não conformes**
- ☐ Menu, opção 2: **lote 7: 24 · lote 8: 40**
- ☐ Benchmark: mediana do NCNN perto de **67 ms**
- ☐ LED ou relé dá **um pulso por lata reprovada**
- ☐ Temperatura estável em uso contínuo (cooler ativo)

na Raspberry, dentro de raspberry/

```
curl localhost:8000/saude
sudo docker compose exec inspecao \
    python scripts/9_benchmark.py
vcgencmd measure_temp      # temperatura
sudo docker compose logs -f # mensagens

# atualizar para a versão nova do Git
cd ~/inspecao-latas && git pull
sh raspberry/iniciar.sh
```

Aceite: /saude com ncnn, 160 latas, ~67 ms no benchmark, um pulso por reprovada, temperatura estável.

Validado no PC e em emulação ARM; a lista acima é o aceite na placa real

Para quem vai desenvolver: sem Docker

terminal, na pasta do projeto

```
python -m venv venv
venv\Scripts\activate
pip install torch==2.14.1 torchvision==0.29.1 ^
  --index-url https://download.pytorch.org/whl/cu130
pip install -r requirements.txt

python scripts/0_verificar_gpu.py # GPU ok?
python scripts/menu.py           # menu
uvicorn api:app --app-dir scripts # API
```

SCRIPT

FAZ

1_capturar

vídeo → imagens

2_anotar

marca as latas

3_separar

treino / validação

4_treinar

gera o best.pt

5_prerotular

marca o resto

6_inspecionar

vídeo com janela

8_exportar_ncnn

formato da Pi

9_benchmark

mede a latência

Sem Docker: venv, PyTorch com CUDA, requirements.txt e scripts numerados. Ajustes em scripts/config.py.

Tudo que se ajusta (classes, limiar, linha, pino) fica em scripts/config.py · no Linux: source venv/bin/activate e \ no lugar de ^

Guardar e publicar as mudanças no GitHub

primeira vez (repositório novo)

```
git init
git add .
git commit -m "Primeira versão"
git branch -M main
git remote add origin LINK-DO-REPOSITORIO
git push -u origin main
```

no dia a dia

```
git status           # o que mudou?
git add .
git commit -m "Ajusta o limiar"
git push
```

LINK-DO-REPOSITORIO: crie o repositório vazio em github.com → **New** e copie o link

.gitignore deixa de fora: `venv/`, `saida/`,
`dataset/yolo/`, `yolo26n.pt`

Sobem: código, Dockerfile e o modelo em `treinos/latas/weights/` (o Docker precisa dele)

Na placa, `git pull` traz a versão nova

Se algo der errado

MENSAGEM OU SINTOMA	O QUE FAZER
<code>docker</code> não é reconhecido	Instale ou abra o Docker Desktop
<code>Cannot connect to the Docker daemon</code>	Espere o Docker Desktop mostrar Engine running
<code>no configuration file provided</code>	O terminal não está na pasta do projeto (<code>cd inspecao-latas</code>)
<code>port is already allocated</code>	Troque "8000:8000" por "8080:8000" no <code>docker-compose.yml</code>
<code>exec format error (Pi)</code>	O sistema gravado é 32 bits: use o Raspberry Pi OS 64-bit
Montagem para sem memória (Pi)	Aumente a swap: <code>CONF_SWAPSIZE=1024</code> em <code>/etc/dphys-swapfile</code>
<code>inspecao.local</code> não encontrado	Use o IP da placa (aparece no roteador)
Câmera 0 não abre (Pi)	Confira com <code>ls /dev/video*</code>

Docker fechado, pasta errada, porta ocupada, Pi 32 bits, falta de memória. Persistiu: `docker compose logs`.

Ainda com erro? Rode `docker compose logs` e copie a mensagem inteira

Onde aprender mais



Documentação Ultralytics

docs.ultralytics.com · predict, train, track, export



Repositório do YOLO

github.com/ultralytics/ultralytics



Vídeos oficiais

youtube.com/@Ultralytics



Guia Raspberry Pi

docs.ultralytics.com/guides/raspberry-pi



Datasets

universe.roboflow.com · kaggle.com/datasets



PyTorch · Docker · FastAPI

pytorch.org · docs.docker.com · fastapi.tiangolo.com

Este projeto: github.com/geanusksama/inspecao-latas

README · COMO_TESTAR.md · scripts numerados na ordem dos passos

O que melhorar na solução



Mais dados

Mais exemplos de lata sem cor (prata lisa), a classe mais difícil



FP16 / INT8

Quantizar só se precisar, medindo a perda de mAP



Validar na placa real

Até aqui: PC e emulação ARM64. Falta a Raspberry Pi 5 física



Acelerador (NPU)

Raspberry Pi AI HAT+ (Hailo) para liberar a CPU

AGRADECIMENTO

Obrigado!

Agradeço à banca técnica e pedagógica do FIT pelo tempo e pela oportunidade de apresentar este projeto.

Francisco Lima

Perguntas e demonstração ao vivo · localhost:8000/docs

github.com/geanusksama/inspecao-latas